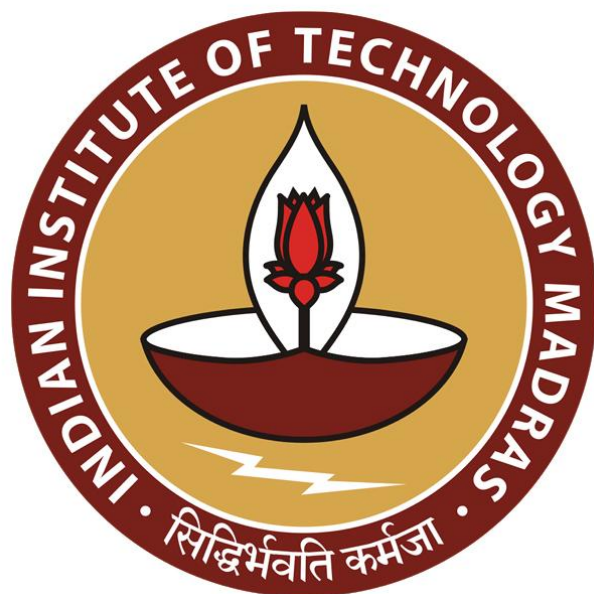


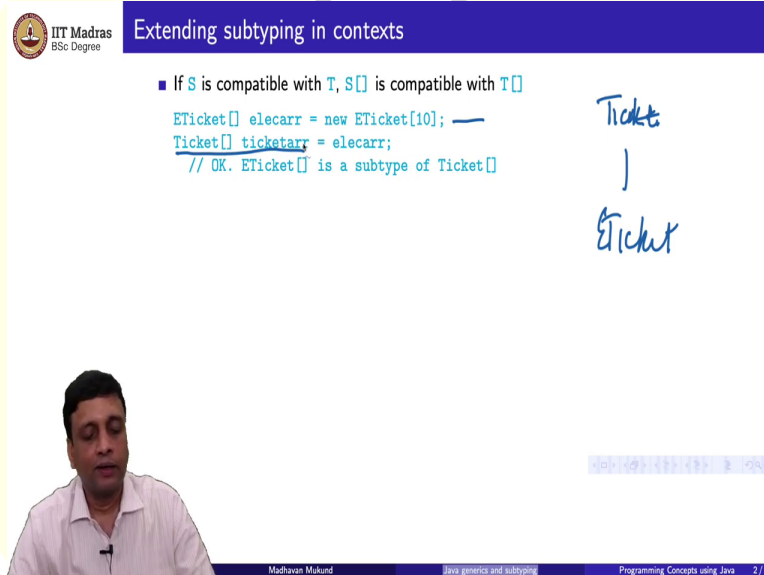


IIT Madras
ONLINE DEGREE

Programming Concepts using Java
Professor Madhavan Mukund
Department of Computer Science
Chennai Mathematical Institute
Indian Institute of Technology Madras
Java Generics and Subtyping

We have seen the type variables allow us to do generic programming in java. So, we can bring in what is called polymorphism over structure. So, structural polymorphism using these types variables. So, now let us see how this generic interacts with other feature in java such as subtyping.

(Refer Slide Time: 0:32)



Extending subtyping in contexts

- If S is compatible with T , $S[]$ is compatible with $T[]$

```
ETicket[] elecarr = new ETicket[10]; —  
Ticket[] ticketarr = elecarr;  
// OK. ETicket[] is a subtype of Ticket[]
```

Handwritten notes:

```
Ticket  
|  
ETicket
```

Video inset: Madhavan Mukund

Slide footer: Madhavan Mukund | Java generics and subtyping | Programming Concepts using Java | 2/9

Extending subtyping in contexts

- If S is compatible with T , $S[]$ is compatible with $T[]$

```
ETicket[] elecarr = new ETicket[10];
Ticket[] ticketarr = elecarr;
// OK. ETicket[] is a subtype of Ticket[]
```

- But ...

```
...
ticketarr[5] = new Ticket();
// Not OK. ticketarr[5] refers to an ETicket!
```



Extending subtyping in contexts

- If S is compatible with T , $S[]$ is compatible with $T[]$

```
ETicket[] elecarr = new ETicket[10];
Ticket[] ticketarr = elecarr;
// OK. ETicket[] is a subtype of Ticket[]
```

- But ...

```
...
ticketarr[5] = new Ticket();
// Not OK. ticketarr[5] refers to an ETicket!
```

- A type error at run time!
- Java array typing is **covariant**
 - If S extends T then $S[]$ extends $T[]$



So, let us first go back to arrays. So, arrays if a type S is compatible with T , in other words, if S extends T S is a subtype of T , then an array of S is also, compatible with an array of T . So, for instance, if we have our usual thing where we have ticket as a superclass, and we have below that we have ETicket as a subclass so, we have ticket an ETicket, then we first define an ETicket[] array.

This is a more specific type then because it is more specific, we can take a ticket array and assign it an array of type ETicket. So, wherever we see a ticket, remember the logic is that wherever we expect a super type variable we can associate a subtype value. So, with a super type array, an

array of type ticket, I am allowed to associate an array of type ETicket because this is the general type compatibility between super types and subtypes for individual objects and it extends to arrays. So, this is fine.

Now, let us see what happens if we directly try to manipulate. So, notice that after this ticketarr and elecarr are both pointing to exactly the same object, which is actually an object of ETicket. And in fact, it was created as an object of ETicket. But because I do not have this information statically, I could write in my program and assignment to the ticket array directly and try to rewrite position 5 in the ticket array by a new ticket.

We remember the position 5 and ticket arrays actually position 5 in this ETicket array. So, it is expecting only ETicket there because that is how this array was declared. Ticket array was just a kind of alias for this. But at run at compile time, Java has no way of checking that at this point, ticket array, which is declared to be an array of type ticket cannot take a value of type ticket in one of its positions.

So, the compiler will let this pass. And of course, at runtime, when we find that this ticket array is actually an alias for an ETicket array, we will get a type error. So, this is the kind of situation that we really want to avoid, which is to get type errors at runtime. And unfortunately, because Java arrays have this compatibility which is called covariance technically in the programming language theory literature.

So, it says that if S extends T, then an array of S extends an array of T. So, this simple-minded covariance is what a programmer would expect, you would expect that I could take something which is a subtype and substitute it in any context. So, the whole idea is that in a context can I substitute a subtype or not and it would make sense that in an array context, you can take a subtype and substitute it. But unfortunately, we have this consequence that we get runtime type errors.

(Refer Slide Time: 3:36)



Generics and subtypes

- Generic classes are not covariant
 - `LinkedList<String>` is not compatible with `LinkedList<Object>`



Madhavan Mukund

java generics and subtyping

Programming Concepts using Java 3 / 9



Generics and subtypes

- Generic classes are not covariant
 - `LinkedList<String>` is not compatible with `LinkedList<Object>`
- The following will not work to print out an arbitrary `LinkedList`

```
public class LinkedList<T>{...}  
  
public static void printlist(LinkedList<Object> l){  
    Object o;  
    Iterator i = l.get_iterator();  
    while (i.has_next()){  
        o = i.get_next();  
        System.out.println(o);  
    }  
}
```



Madhavan Mukund

java generics and subtyping

Programming Concepts using Java 3 / 9

सिद्धिर्भवति कर्मजा

- Generic classes are not covariant
 - `LinkedList<String>` is not compatible with `LinkedList<Object>`
- The following will not work to print out an arbitrary `LinkedList`

```
public class LinkedList<T>{...}

public static void printlist(LinkedList<Object> l){
    Object o;
    Iterator i = l.get_iterator();
    while (i.has_next()){
        o = i.get_next();
        System.out.println(o);
    }
}
```
- How can we get around this limitation?



Madhavan Mukund

Java generics and subtyping

Programming Concepts using Java 3 / 9

So, in order to avoid this generic classes, when they were introduced, remember the generic classes were introduced into Java after the language was defined. So, by this time people had already noticed and observed this covariance problem with arrays. So, when generic types were introduced, the conscious decision was taken not to have this type compatibility. So, if we had type compatibility, then if you substitute `T` with `object`, which is of course the root of our type hierarchy.

Then it should be a super type of a linked list in this case with any other type in our hierarchy. This would certainly be true if I had an array of `object`, it would allow me to substituted by an array of strings in a concrete situation. But in the generic case, this is not true. So, a linked list of a subtype is not compatible the linked list of a supertype. So, in particular, right at the beginning, we wrote this generic `find`, which took two inputs an array of `object` and a value of type `object`.

And try to check whether that value occurred in the array or not. So, if we wanted to print out a list, which is defined using this parameterised linked list type. We cannot use `object` as the Super type to make this a generic function. So, we cannot write a function like this because linked lists of string or linked list of manager or link list of date will not be allowed to be substituted for this. The only argument that I can pass to this function is actually of type link list object. So, the way to get around this of course is to write generic functions, which we have already seen.

(Refer Slide Time: 5:17)



Generic methods

- As we have seen, we can make the method generic by introducing a type variable

```
public class LinkedList<T>{...}  
  
public static <T> void printlist(LinkedList<T> l){  
    Object o;  
    Iterator i = l.get_iterator();  
    while (i.has_next()){  
        o = i.get_next();  
        System.out.println(o);  
    }  
}
```



Madhavan Mukund

Java generics and subtyping

Programming Concepts using Java 4/9



Generic methods

- As we have seen, we can make the method generic by introducing a type variable

```
public class LinkedList<T>{...}  
  
public static <T> void printlist(LinkedList<T> l){  
    Object o;  
    Iterator i = l.get_iterator();  
    while (i.has_next()){  
        o = i.get_next();  
        System.out.println(o);  
    }  
}
```

- **<T>** is a type quantifier: For every type T, ...



Madhavan Mukund

Java generics and subtyping

Programming Concepts using Java 4/9

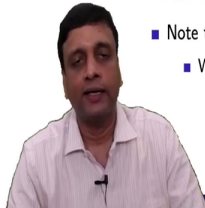
सिद्धिर्भवति कर्मजा

- As we have seen, we can make the method generic by introducing a type variable

```
public class LinkedList<T>{...}

public static <T> void printList(LinkedList<T> l){
    Object o;
    Iterator i = l.get_iterator();
    while (i.has_next()){
        o = i.get_next();
        System.out.println(o);
    }
}
```

- <T> is a type quantifier: For every type T, ...
- Note that T is not actually used inside the function
 - We use Object o as a generic variable to cycle through the list



Madhavan Mukund

Java generics and subtyping

Programming Concepts using Java 4 / 9

So, what we can do is we can make the method generic by introducing a type variable. So, we can introduce a type variable here, before the return type. And what this says is that this function is now parameterized by the type, and this type variable is now being used over here. So, this is a generic function as we saw in the previous module on generics. And this is precisely saying that this angle bracket T is what we call in logic a quantifier.

It says for every type T this function works. So, for every type capital T, this is a function which will print a list whose underlying objects are of type T. So, this is of course one standard way of doing it which we have already seen. But notice that after we pass in the T, we are not using T anywhere inside the function. And we are also, not returning any value which depends on T. So, we have a free choice of type to enter the function.

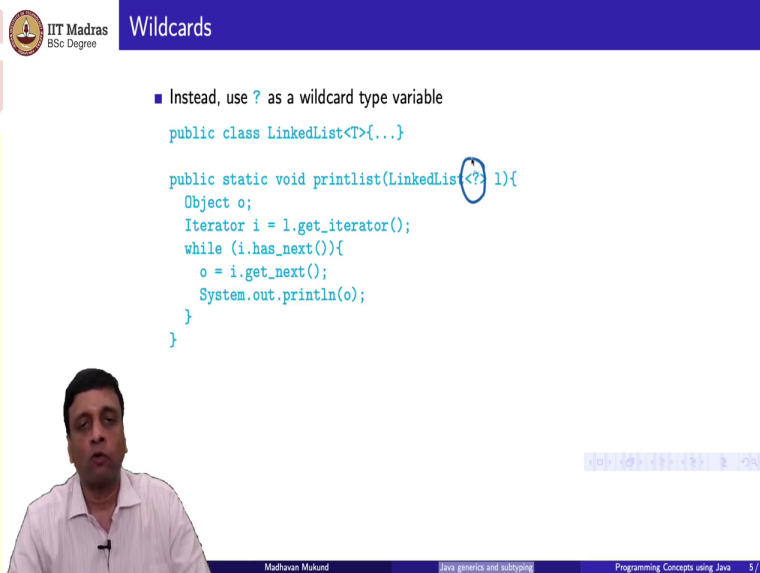
And once we enter the function, what we are really using is the other part of the Java hierarchy, which says that an object of type of an O of type object can be used to hold any value. So, we are really taking this get next function in our iterator. So, what is this doing? It is using an iterator. It is getting an iterator for this list. And it is running through the list, and it is printing out each object in the list.

But to store the object before we print it, we are using this fact that in the Java type hierarchy capital O object is sitting at the root of the hierarchy, so, it can stand for any value. And at this point, we are not going to manipulate it, we are not going to call any functional on we just want

to print it. So, we just pass it to the print function and it gets printed. So, in this case, we are not using this T anywhere.

So, this is a very special kind of function where the type quantifier allows us to instantiate any type in the argument. But once we instantiate it, we do not need to remember what type it is there is no dependence of that T anywhere else inside the function either inside internal variables or in the return type.

(Refer Slide Time: 7:32)



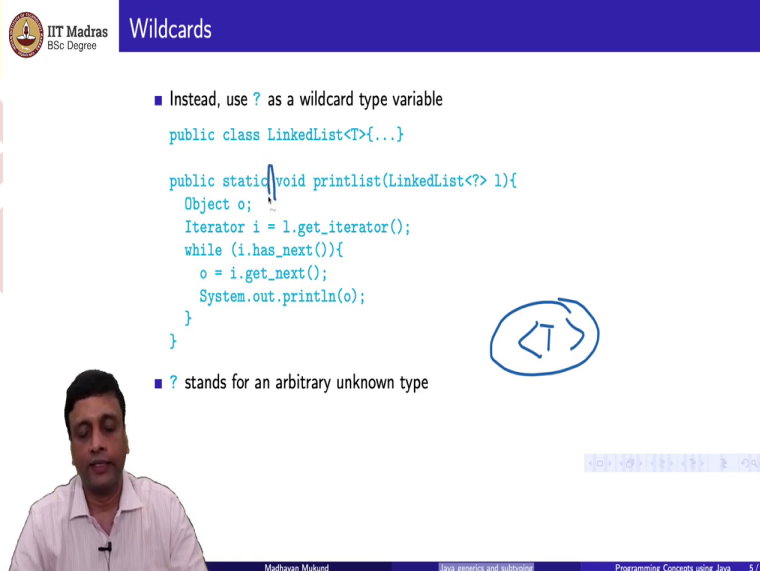
Wildcards

- Instead, use ? as a wildcard type variable

```
public class LinkedList<T>{...}

public static void printlist(LinkedList<?> l){
    Object o;
    Iterator i = l.get_iterator();
    while (i.has_next()){
        o = i.get_next();
        System.out.println(o);
    }
}
```

Madhavan Mukund | Java generics and subtyping | Programming Concepts using Java | 5/9



Wildcards

- Instead, use ? as a wildcard type variable

```
public class LinkedList<T>{...}

public static void printlist(LinkedList<?> l){
    Object o;
    Iterator i = l.get_iterator();
    while (i.has_next()){
        o = i.get_next();
        System.out.println(o);
    }
}
```

- ? stands for an arbitrary unknown type

Madhavan Mukund | Java generics and subtyping | Programming Concepts using Java | 5/9

- Instead, use ? as a wildcard type variable

```
public class LinkedList<T>{...}

public static void printlist(LinkedList<?> l){
    Object o;
    Iterator i = l.get_iterator();
    while (i.has_next()){
        o = i.get_next();
        System.out.println(o);
    }
}
```

- ? stands for an arbitrary unknown type
- Avoids unnecessary type variable quantification when the type variable is not needed elsewhere



So, when such a situation happens, then the name T is actually not important. So, we do not really need to remember that it was of type T because T is not being used. So, Java has another mechanism to deal with this called a wild card type. So, here what we can do is we can do away with this, notice that there is no type quantifier here. And instead of T we use this special symbol question mark. So, this is an alternative way of writing an arbitrary type.

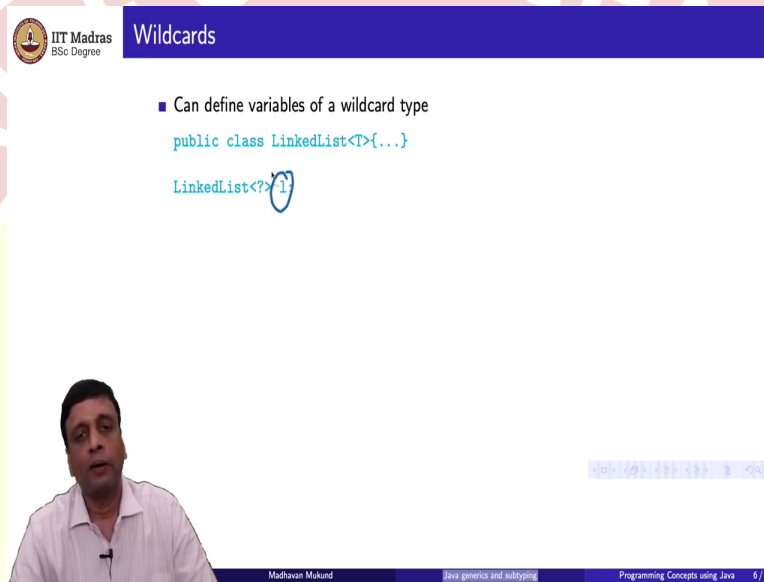
Now, of course, when we write question mark, it does not have a name. So, I could not use any values of type question mark, I cannot return Something or type question mark with there is no way to compare. But if this is the kind of situation where I am not going to make use of that value anywhere inside the function, then a question mark is fine. So, question mark stands for an arbitrary unknown type.

So, it is not very different from saying that you quantifier O we need earlier we said that this said for all T do Something. So, this is really saying that T is an arbitrary unknown type, but we remember T. So, the T that goes in is the T that is used somewhere else is a T that is returned. So, there is a kind of consistency with the usage of T. But in situations where that type is not being used anywhere we do not need to give it a name.

And what we save on is this angle bracket notation “<>”. We do not have to write anything here to specify that it is a quantified type. We just implicitly do it by using this question mark. So, this question mark or this generic wildcard allows us to avoid an unnecessary type variable whenever

the type variable is not needed elsewhere in the function. So, this is just a small shortcut, but it saves you having to write a type variable if you are feeling uncomfortable about writing that it does not. So, the two are completely equivalent. You can just write the quantifier, you can write angle bracket T and use T throughout the function or if you do not need T anywhere in the function. In that special case, you can use this question mark “<?””.

(Refer Slide Time: 9:26)



IIT Madras BSc Degree

Wildcards

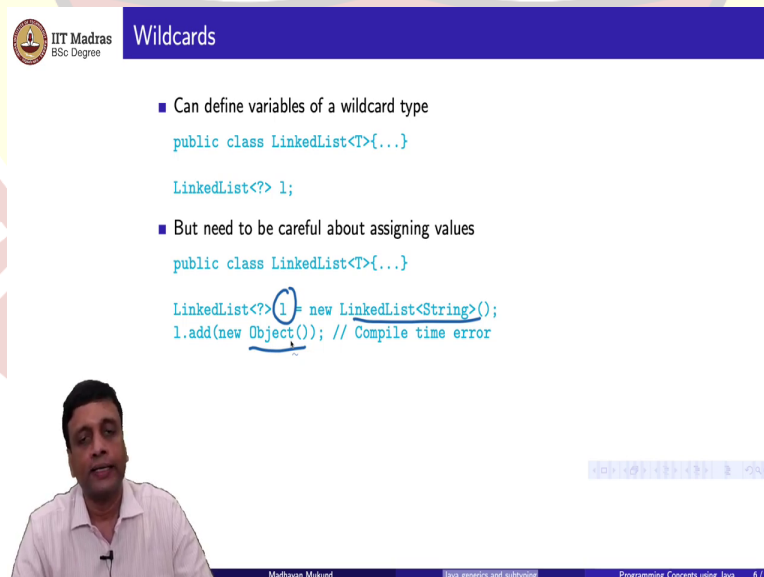
- Can define variables of a wildcard type

```
public class LinkedList<T>{...}  
LinkedList<?> l;
```

Madhavan Mukund

java generics and subtyping

Programming Concepts using Java 6/9



IIT Madras BSc Degree

Wildcards

- Can define variables of a wildcard type

```
public class LinkedList<T>{...}  
LinkedList<?> l;
```

- But need to be careful about assigning values

```
public class LinkedList<T>{...}  
LinkedList<?> l = new LinkedList<String>();  
l.add(new Object()); // Compile time error
```

Madhavan Mukund

java generics and subtyping

Programming Concepts using Java 6/9

- Can define variables of a wildcard type

```
public class LinkedList<T>{...}

LinkedList<?> l;
```

- But need to be careful about assigning values

```
public class LinkedList<T>{...}

LinkedList<?> l = new LinkedList<String>();
l.add(new Object()); // Compile time error
```

- Compiler cannot guarantee the types match



So, it turns out that you can actually define variables of this type. So, you can actually say that L is a linked list of some arbitrary but unknown type (LinkedList<?>). But for obvious reasons, it is not really possible to add anything concrete to this. So, if I take a linked list of this type and then I say that it is at runtime a list of string. The problem is that we do not have any way of knowing what the type is. So, the compiler will not allow you to add other objects to this because it does not know whether this compatible or not because you said it is a list of any type.

So, it really does not have a concrete type associated with it at compile time. So, this is the problem. So, compiler cannot guarantee that types match. So, the moral of the story is that you can use wildcard types quite easily as arguments to functions, where the type information of the arbitrary type is not required anywhere in the function. But do not use it in general in order to generate variables and assign values to them.

(Refer Slide Time: 10:28)



Bounded wildcards

- Suppose `Circle`, `Square` and `Rectangle` all extend `Shape`

S extends T



Madhavan Mukund

Java generics and subtyping

Programming Concepts using Java 7/9



Bounded wildcards

- Suppose `Circle`, `Square` and `Rectangle` all extend `Shape`
- `Shape` has a method `draw()`
- All subclasses override `draw()`
- Want a function to draw all elements in a list of `Shape` compatible objects

```
public static void drawAll(LinkedList<? extends Shape> l){  
    Object o  
    Iterator i = l.get_iterator();  
    while (i.has_next()){  
        o = i.get_next();  
        o.draw();  
    }  
}
```



Madhavan Mukund

Java generics and subtyping

Programming Concepts using Java 7/9

So, we do not have to have free generic type while wildcard types, we can have what are called bounded wildcards. So, we saw this in the context of type variables, we said we could write something like `S extends T`. So, we could compare one variable to another variable and constraint it. So, `S` and `T` are two different types, but they are related by the fact that `S` must be a subtype of `T`.

So, now, suppose we have a super type `Shape`, which has three subtypes, `Circle`, `Square`, and `Rectangle`. And let us imagine that the super type `Shape` has a method defined to draw a shape.

And it so, happens that each of the subtypes, circle square and rectangle actually override the draw. So, if you ask a circle to draw itself, it will use a more specialized method, similarly with the square similarly with a rectangle.

So, now, the typical kind of thing that we would like to do is process an array which has multiple types of shapes, and print all of them using this or draw all of them. So, how do we describe a list which can hold any type which is compatible with shape. So, we do not want it to be a linked list of anything. But we want it to be a linked list or something which is compatible with shape. Now, in the same way that earlier, we were just printing out an object.

When we said the print list command, in this, we have a draw all command. And all it does is it takes an object and it draws it. So, we can use a simple variable of type object in order to run through this list. So, all we need to specify is the list itself is well type. So, that draw comes. So, this is where now we can use this generic wildcard. But we can bound it by saying that it cannot be any old type, it must be a type which extends shape.

So, we do not care what it is, we do not need to know its name, because we are not going to use that type anywhere inside. But at the same time, we have constrained it so, that it extends shape. So, it is the same way that we can take two concrete type variables and constrain one to be a subtype of another, we can take this wildcard and constrain it to be a subtype of Some other type.

(Refer Slide Time: 12:41)

- Copying a `LinkedList`, using a wildcard

```
public static <? extends T,T>
void listcopy (LinkedList<?> src,
               LinkedList<T> tgt){
    Object o;
    Iterator i = src.get_iterator();
    while (i.has_next()){
        o = i.get_next();
        tgt.add(o);
    }
}
```



Navigation icons: back, forward, search, etc.

Madhavan Mukund

Java generics and subtyping

Programming Concepts using Java 8/9

Now, we can also use this in the context of type variables. So, that was an example where we constrained it with respect to a concrete type shape. But remember our array copy where we took two arrays source and target and we said that we can copy a source array to a target array so long as the target is a super type of the source. So, we can do a similar thing, for example, with our generic link list.

So, we would like to do a list copy, which will take a source list (`LinkedList<T> src`) and a target list (`LinkedList<T> tgt`). And now what do we want? Well, we want that this list is a subtype of this list. So, the target list must be a super type of the source list or the Source list must be a subtype of the target list. So, in the signature of this function, we add this constraint which said that the second argument has some arbitrary type `T` and the first one has some unknown type question mark.

But this question mark extends `T`. So, this is just I could have equally well written here `S` and written `S` here. But it is just to show that you can use this question mark, also in conjunction with type variables. So, you can use type variables, you can use a question mark, you can use both. And again, here, the point is that in this array copy, we are going to learn this iterator through this list.

And we are just going to pull out the object from one list. And we are going to add it to the other list. So, we are not doing anything else with `O`. So, we do not need any specific properties of `O`.

So, we do not really care what that question mark stands for. Now, we can interestingly, we can reverse this constraint. So, instead of saying that the first type is a subtype of the second type which is what this constraint is saying. We can also, say that the second type is a super type of the first type.

(Refer Slide Time: 14:27)

Bounded wildcards

- Copying a `LinkedList`, using a wildcard
- Can reverse the constraint, using `super`

```
public static <? extends T>
void listcopy (LinkedList<?> src,
               LinkedList<T> tgt){
    Object o;
    Iterator i = src.get_iterator();
    while (i.has_next()){
        o = i.get_next();
        tgt.add(o);
    }
}
```

```
public static <T, ? super T>
void listcopy (LinkedList<T> src,
               LinkedList<?> tgt){
    Object o;
    Iterator i = src.get_iterator();
    while (i.has_next()){
        o = i.get_next();
        tgt.add(o);
    }
}
```

Madhavan Mulund Java generics and subtyping Programming Concepts using Java 8 / 9

So, we can now say that the first list, the source list has type T. And now the target list is any type which is above T. So, just like extends you can use super as a constraint. So, this is a new thing, which we have not seen before, but this is just to show that it is symmetric. So, in this example, it does not really matter which way you do it.

But in case you need to use T for one of the two sides, you can decide which side you want to use T and for the other side you can use question mark remember that we need to use T in case T appears somewhere in our code. Now supposing it is natural for the first argument to appear in our code, then we would like to give a name for the first argument. And maybe the second argument does not need to be named.

On the other hand, it could be that the second argument needs to be named, in which case, the first argument may not need to be named. Of course, if both need to be named, then you would use to type variables, which is perfectly fine. But you can have this kind of flexibility to decide

which of the type variables can become a wildcard in case you have this constraint between two of them.

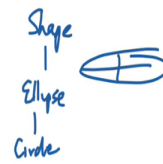
(Refer Slide Time: 15:29)



Summary

- Java generics are not covariant, unlike arrays
- Cannot substitute `Object` for `T` to get most general type
- Instead, use type quantification `<T>` or wild card type variable `?`
- Wild card can be used wherever the type `T` is not required within the function
 - When `T` is not needed for return type, or to declare local variables
- Wild cards can be bounded
 - `LinkedList<? extends T>`
 - `LinkedList<? super T>`

transitive



Madhavan Mukund

Java generics and subtyping

Programming Concepts using Java 9/9

So, to summarize, what we have seen is that Java arrays are covariant. If `S` is a subtype of `T`, then an array of `S` is a subtype of an array of `T`. And this potentially leads to runtime type errors because we can have a situation where we accidentally try to assign a variable of a subtype to an array which is declared of a super type through this aliasing that we saw with `ETickets` and `Tickets`.

So, in order to get around this, the generic types are not covariant. So, if we have a parameterized data structure, we cannot say that substituting the parameter with `Object` creates the most general type of that parameterized type. So, a linked list of `Object` is not a super type of a linked list of `String`, even though `Object` individually is a root super type of `String`. So, instead we saw that we can use type quantification in order to make things generic.

But we have also now seen that there is a special situation where we can use this wildcard question mark in place of an explicit type variable. So, wildcard, in general can be used wherever the type `T` is not needed explicitly in the rest of the function, it is only needed to specify a kind of

constraint or unconstrained access to the argument. But it is not using the return value and it is not using any local variables.

Then we do not really need to remember what T is when we enter the function. So, we can then freely use the question mark to indicate this freedom of choice of type and proceed. And like normal type variables, we can type we can bound our wildcards using either extends or super so, we can describe the fact that we want an arbitrary type.

Which is a subtype of a given type for an arbitrary type which is super type and then all these things when we say extends and super, these are what are called transitive it is not saying that these are immediately subtype it is not that extends when we write extends in the class definition, it is one step when we say that circle extends shape. It means shape is one level above circle in the hierarchy.

But supposing we had ellipse extends shape and then circle extends ellipse supposing we had shape and then we have an ellipse. Then we might have a circle as a special kind of an ellipse remember, an ellipse is something which has a major axis and a minor axis. So, circle can be thought of as an ellipse, where the major and minor axis is the same. So, in this case, technically speaking, when you write circle we would say circle extends, ellipse and ellipse extend shape. But in this kind of a type, expression, circle extend shape is true. So, it is a transitive requirement. Similarly, shape is a superclass of circle is true. In this transitive requirement.